

Deployment Guide

Project: RoomRadar QC

1. Purpose

This deployment guide explains how to install, configure, build, and run the RoomRadar QC project.

RoomRadar QC is a web application with a React/Vite frontend, Flask/Python backend, MySQL database, and Firebase authentication for admin-related backend features.

2. Technology Stack

Component	Technology
Frontend	React with Vite
Backend	Python Flask
Database	MySQL
Authentication	Firebase Admin SDK
API Format	JSON
Package Manager	npm for frontend, pip for backend

3. JDK/JRE and Tomcat Requirement

This project does not use Java, JDK/JRE, or Apache Tomcat.

RoomRadar QC is not an Eclipse/Tomcat web application. It is a React and Flask web application.

Requirement	Status
JDK/JRE version	Not applicable

Requirement	Status
Tomcat version	Not applicable
Eclipse project import	Not applicable
Android Studio/XCode	Not applicable

Because the project does not use Java or Tomcat, there is no JDK/JRE or Tomcat version required to build or run the application.

4. Required Software

To run the project locally, install the following software:

Software	Purpose
Python 3	Runs the Flask backend
pip	Installs Python backend dependencies
Node.js	Runs the React/Vite frontend
npm	Installs frontend dependencies
MySQL Server	Stores room, semester, and schedule data
Git	Clones or manages the project repository
Firebase service account JSON	Required for Firebase Admin authentication

5. Backend Setup

5.1 Navigate to the Backend Folder

```
cd backend
```

5.2 Create a Python Virtual Environment

On Mac/Linux:

```
python3 -m venv venv
source venv/bin/activate
```

On Windows:

```
python -m venv venv
venv\Scripts\activate
```

5.3 Install Backend Dependencies

On Mac/Linux:

```
python3 -m pip install -r requirements.txt
```

On Windows:

```
pip install -r requirements.txt
```

5.4 Firebase Admin Credential Setup

The backend uses Firebase Admin SDK for authentication checks on admin routes. The Firebase service account JSON file is required locally, but it should not be committed to GitHub for security reasons.

Place the Firebase service account JSON file inside the `backend/` folder.

Expected file:

```
backend/roomradar-qc-firebase-adminsdk-fbsvc-05922392f1.json
```

The backend initializes Firebase Admin in `app.py` before registering routes and starting the Flask application.

```
cred = credentials.Certificate("roomradar-qc-firebase-adminsdk-fbsvc-05922392f1.json")
firebase_admin.initialize_app(cred)
```

If the JSON file is missing, admin authentication features may not work.

6. Database Setup

RoomRadar QC uses a MySQL database. The backend connects to MySQL using database settings from `config.py`.

6.1 Create the Database and Tables

Use the SQL files inside the `database/` folder.

If the database does not exist yet, run:

```
SOURCE install_database.sql;
```

If the database already exists and only the tables need to be created, run:

```
USE roomradar_qc;  
SOURCE create_tables.sql;
```

The project uses three main tables:

Table	Purpose
<code>semesters</code>	Stores uploaded semester names
<code>rooms</code>	Stores room codes, building codes, and room capacity
<code>schedules</code>	Stores class schedule records connected to rooms and semesters

The backend also contains a `create_tables()` function in `models.py` that creates these tables automatically if they do not already exist.

7. Running the Backend

From the `backend/` folder, run:

On Mac/Linux:

```
python3 app.py
```

On Windows:

```
python app.py
```

The backend usually runs on:

```
http://127.0.0.1:5000
```

or:

```
http://localhost:5000
```

Use the exact URL printed in the terminal after Flask starts.

Example terminal output:

```
* Running on http://127.0.0.1:5000
```

8. Frontend Setup

8.1 Navigate to the Frontend Folder

From the project root:

```
cd frontend
```

8.2 Install Frontend Dependencies

```
npm install
```

8.3 Run the Frontend Development Server

```
npm run dev
```

The frontend usually runs on:

```
http://localhost:5173
```

If port `5173` is already in use, Vite may automatically choose another port, such as:

```
http://localhost:5174
```

Use the URL shown in the terminal after running `npm run dev`.

Example terminal output:

```
Local: http://localhost:5174/
```

9. API URL Configuration

The frontend currently uses local backend API URLs such as:

```
http://127.0.0.1:5000
```

or:

```
http://localhost:5000
```

This works for local development. If the backend is deployed remotely, these API URLs must be changed to the deployed backend address.

Example local URL:

```
http://127.0.0.1:5000/api/rooms/search
```

Example deployed URL:

```
https://roomradar-backend.example.com/api/rooms/search
```

10. Backend API Endpoints

10.1 Room Search Endpoint

GET /api/rooms/search

Example:

http://127.0.0.1:5000/api/rooms/search?day=Tuesday&starttime=12:30&endtime=14:30

This endpoint returns rooms that do not have a scheduled class conflict during the selected time range.

10.2 Room Schedule Endpoint

GET /api/rooms/{room_id}/schedule

Example:

http://127.0.0.1:5000/api/rooms/1/schedule?day=Tuesday

This endpoint returns the selected room's schedule for the chosen day.

10.3 Admin Semester Upload Endpoint

POST /api/admin/semester

This endpoint allows an authenticated admin user to upload semester schedule data.

The request requires a Firebase authentication token in the `Authorization` header.

Example:

Authorization: Bearer <firebase_id_token>

11. Build Instructions

11.1 Build the Frontend

From the `frontend/` folder:

```
npm run build
```

This creates a production build in:

```
frontend/dist/
```

The `dist/` folder is generated by Vite and should not be committed unless specifically required for deployment.

11.2 Backend Build

The Flask backend does not require a separate build step. It runs directly with Python after dependencies are installed.

12. Third-Party Libraries

12.1 Backend Libraries

Backend dependencies are listed in:

```
backend/requirements.txt
```

They may include libraries such as:

- Flask
- Flask-CORS
- mysql-connector-python

- firebase-admin

Install them with:

```
python3 -m pip install -r requirements.txt
```

12.2 Frontend Libraries

Frontend dependencies are listed in:

```
frontend/package.json
```

They may include libraries such as:

- React
- Vite
- Material UI
- React Router

Install them with:

```
npm install
```

13. Local Development Startup Order

To run the full project locally:

Terminal 1: Start Backend

```
cd backend  
source venv/bin/activate  
python3 app.py
```

Backend URL:

```
http://127.0.0.1:5000
```

Terminal 2: Start Frontend

```
cd frontend  
npm run dev
```

Frontend URL:

```
http://localhost:5173
```

or the port shown by Vite.

14. Verification Steps

After starting both servers:

1. Open the frontend URL in the browser.
2. Go to the Find Available Rooms page.
3. Select a day, start time, end time, and building.
4. Click Search Rooms.
5. Confirm that room cards appear.
6. Click View Room Schedule.
7. Confirm that the Room Details page loads the room schedule.
8. Test admin login and CSV upload if Firebase credentials are configured.

15. Common Issues and Fixes

15.1 Frontend Port Already in Use

If Vite says:

```
Port 5173 is in use, trying another one...
```

then use the new URL shown in the terminal, such as:

```
http://localhost:5174
```

15.2 Backend API Not Reached

If the frontend cannot reach the backend, check that Flask is running and confirm the backend URL printed in the terminal.

If Flask prints:

```
http://127.0.0.1:5000
```

then the frontend API calls should use that address.

15.3 Missing Firebase Service Account

If the Firebase service account JSON file is missing, admin authentication may fail.

The file should be placed in the `backend/` folder but should not be committed to GitHub.

15.4 MySQL Connection Error

If the backend cannot connect to MySQL:

1. Confirm MySQL is running.
2. Confirm the database exists.
3. Confirm the database username and password in `config.py`.
4. Confirm the database host and port are correct.

16. Summary

RoomRadar QC does not require JDK/JRE or Tomcat because it is not a Java web application. The project runs using a React/Vite frontend, Flask/Python backend, MySQL database, and Firebase Admin authentication.

To run locally:

1. Start the Flask backend from the `backend/` folder.

2. Start the React frontend from the `frontend/` folder.
3. Open the frontend URL shown by Vite.
4. Use the application to search for available rooms and view room schedule details.